```python
import random
import copy
from time import time
import math
import numpy as np
import matplotlib.pyplot as plt


################################
# 根据各城市的location，计算出各城市
# 间的距离矩阵R
def distCal(location):
    nc,dumm = np.shape(location)
    R = np.zeros([nc,nc])
    for i in range(nc):
        for j in range(i+1,nc):
            R[i,j] = np.sqrt((np.asarray((location[i,:]-location[j,:]))**2).sum
            ())
    R = R + R.T
    return R


################################
# 产生N个染色体的初始群体,保存在pop里
# 每个染色体代表TSP问题的某一个解（即所有城市都经过一次的轨迹）
def initPop(N,nc):
    pop = np.zeros([N,nc])
    for i in range(N): #使用随机函数生成N个染色体
        lst=list(range(1,nc))
        random.shuffle(lst)
        pop[i,range(1,nc)] = lst
    pop[range(N),0]=0  #所有染色体都从城市0开始，最后回到城市0.
    return pop
```

N

nc

0~9

1~10

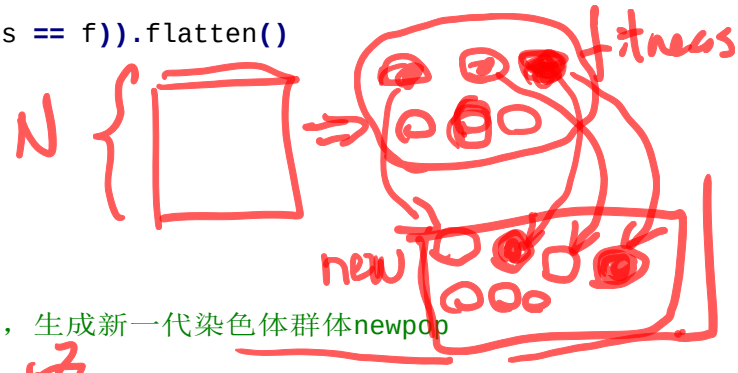
```python


################################
#根据距离矩阵R ，计算pop中每个染色体所代表的轨迹的长度
def calLen(pop,R):
    N,nc=np.shape(pop)
    trajLength = np.zeros([N,1])
    for i in range(N):
        gene = pop[i,:]
        trajLength[i] = calLen4oneGene(gene,R)
    return trajLength
def calLen4oneGene(gene,R):
    disLst = [R[gene[np.mod(j,nc)].astype(np.uint8),gene[np.mod(j+1,nc)].astype
    (np.uint8)] for j in range(nc)]
    return np.sum(disLst)


################################
# 根据所有染色体分别的轨迹长度，计算各个染色体的适应值fitness
```

pop fitness

个体

```python
def calFitness(trajLength):
    fitness = trajLength
    return fitness

##################################
# 根据染色体群体pop已经对应的适应值fitness，
# 找到最高的适应值f，以及对应的染色体bst和其在pop中的编号/下标ind
def findBest(pop,fitness):
    f = np.max(fitness)
    ind = np.asarray(np.where(fitness == f)).flatten()
    bst = pop[ind,:]
    return [bst,f,ind]

def satified(fitness):
    return 0

##################################
# 根据染色体的适应值，按照一定的概率，生成新一代染色体群体newpop
def chooseNewP(pop,fitness):
    newpop = pop
    return newpop

##################################
# 根据交叉概率pc，以及各染色体的适应值fitness，通过交叉的方式生成新群体
# #SelfAdj = 1时为自适应，否则取固定的交叉概率pc
def crossPop(pop,pc,fitness,SelfAdj):
    pass

##################################
# 父染色体partner1,partner2，通过交叉方式
# 生成两个子染色体child1,child2
def genecross(partner1,partner2):
    length = len(partner1)
    # global R
    # distGene1 = [R[partner1[np.mod(i,length)].astype(np.uint8),partner1[np.mod(i+1,length)].astype(np.uint8)] for i in range(length)]
    # distGene2 = [R[partner2[np.mod(i,length)].astype(np.uint8),partner2[np.mod(i+1,length)].astype(np.uint8)] for i in range(length)]
    # div1 = [R[partner1[np.mod(i,length)].astype(np.uint8),:].sum() for i in range(length)]
    # div2 = [R[partner2[np.mod(i,length)].astype(np.uint8),:].sum() for i in range(length)]
    # distPercGene1 = [distGene1[i]/div1[i] for i in range(length)]
    # distPercGene2 = [distGene2[i]/div2[i] for i in range(length)]
    idx1=0
    idx2=0
    while idx1==idx2:
        idx1 = random.randint(0,length-1)
        idx2 = random.randint(0,length-1)
    ind1 = min(idx1,idx2)
    ind2 = max(idx1,idx2)
```

1./trajlength



选择.

```python
    child1 = copy.copy(partner1)
    child2 = copy.copy(partner2)

    tem1 = copy.copy(child1[ind1:ind2])
    tem2 = copy.copy(child2[ind1:ind2])

    if (tem1==tem2).all():
        return [child1,child2]
    if set(tem1)==set(tem2):
        child1[ind1:ind2] = tem2
        child2[ind1:ind2] = tem1
        return [child1,child2]

    temdff1 = set(tem1).difference(set(tem2))
    temdff2 = set(tem2).difference(set(tem1))

    for i in range(len(temdff1)):
        child1[np.where(child1==list(temdff2)[i])]=list(temdff1)[i]
        child2[np.where(child2==list(temdff1)[i])]=list(temdff2)[i]

    child1[ind1:ind2] = tem2
    child2[ind1:ind2] = tem1
    return [child1,child2]

##################################
# 根据变异概率pw，以及各染色体的适应值fitness，通过变异的方式生成新群体
# #SelfAdj = 1时为自适应，否则取固定的变异概率pw
def mutPop(pop,pw,fitness,SelfAdj):
    N,nc=np.shape(pop)

    fm = max(fitness)
    fa = np.mean(fitness)
    k3 = pw
    k4 = pw
    for i in range(N):
        rval = random.random()
        if SelfAdj==1:      # 自适应，参考149页
            if fitness[i]>fa:
                pw = k3*(fm-fitness[i]) / (fm-fa)
            else :
                pw = k4
        if rval > pw:
            continue
        pop[i,:] = np.asarray(mutDistGene(pop[i,:]))


# 在gene的nc个城市中，根据距离值找出最不合理的城市valm以及其下标indx
# 把该城市valm从原有轨迹中抽取出来（其两边的城市直接相连），并把valm城市
# insert到列表的indy的位置。
def mutDistGene(gene):
    global R
    global randAccordDist
    nc = len(gene)
    distGene = [R[gene[np.mod(i,nc)].astype(np.uint8),gene[np.mod(i+1,nc)].
```

TSP

多样化.

```
        astype(np.uint8)] for i in range(nc)]
        #gene中相邻城市之间的距离,gene[0]和gene[1]的距离保存在distGene[0]中
        distGeneR= [R[gene[np.mod(i,nc)].astype(np.uint8),gene[np.mod(i-1,nc)].
        astype(np.uint8)] for i in range(nc)]
        #gene中相邻城市之间的距离,gene[0]和gene[nc-1]的距离保存在distGeneR[0]中
        distGene = list(np.asarray(distGene) + np.asarray(distGeneR))
        divDist = [R[gene[np.mod(i,nc)].astype(np.uint8),:].sum() for i in range(nc
        )]
        distPercGene = [distGene[i]/divDist[i] for i in range(nc)]

        indx = np.argmax(distPercGene)
        listgene = list(gene)
        valm = int(listgene.pop(indx))

        alist=range(nc)
        clist = [str(alist[i]) for i in range(nc)]
        rList = copy.copy(R[valm, :])
        maxrList = max(rList)
        rList[valm] = 10**10        #原数值为0，换成一个非常大（足够大）的数值
        try:
            rList = (np.exp(maxrList/rList)).astype(np.uint64)
        except:
            print('df')
        rList[valm] = 0             #以便使得valm不会被重复选择

        ind1 = ind2 = valm
        while ind2==valm:
            if randAccordDist == 1:
                dumm1,dumm2,ind1,ind2 = findTwoMaxRandom(list(rList))
            else:
                dumm1,dumm2,ind1,ind2 = findTwoMax(list(rList))

        inda = listgene.index(ind1)
        indb = listgene.index(ind2)
        if np.mod(inda-indb,nc) <= np.mod(indb-inda,nc):
            indy = np.mod(inda,nc)
        else:
            indy = np.mod(inda+1,nc)

        listgene.insert(indy,valm)
        if listgene[0]!=0:
            cycleList(listgene)
        return listgene

    # 旋转队列lst，使得数值0在lst[0]的位置
    def cycleList(lst):
        temlst = []
        while lst[0]!=0:
            temlst.append(lst.pop(0))
        lst.extend(temlst)


    def findTwoMax(lst):
        ind1 = np.argmax(lst)
```

```python
    max1 = lst[ind1]

    lst[ind1] = min(lst)

    ind2 = np.argmax(lst)
    max2 = lst[ind2]
    return [max1,max2,ind1,ind2]

# 按照一定的规则，使得lst的最大和次最大的两个数值及其索引被输出的概率最大
# lst每个元素必须是正数,表示权值
def findTwoMaxRandom(lst):
    indxList = [str(i) for i in range(len(lst))]
    ind1 = np.argmax(lst)
    max1 = lst[ind1]

    lst[ind1] = min(lst)

    ind2 = int(weight_choice(indxList,lst))     # 概率
    max2 = lst[ind2]

    return [max1,max2,ind1,ind2]


# lst: 待选取序列
# weight: lst对应的权重序列
def weight_choice(lst, weight):
    new_lst = []
    for i, val in enumerate(lst):
        new_lst.extend(val * weight[i])
    return random.choice(new_lst)




##################################
# 根据染色体traj所规定的轨迹，以及各城市的位置locations，画出该轨迹
def drawTSP(locations, traj):
    nc,dumm = np.shape(locations)
    #plot(0,0,'.') hold on plot(100,100,'.')
    for i in range(nc):
        indP = int(traj[i])
        strPx = locations[indP, 0]
        strPy = locations[indP, 1]
        endPx = locations[int(traj[np.mod(i+1,nc)]), 0]
        endPy = locations[int(traj[np.mod(i+1,nc)]), 1]
        plt.plot([strPx,endPx],[strPy,endPy],color="blue",marker="o",markersize=14,markerfacecolor="yellow")
        plt.text(strPx,strPy,indP,fontsize=20,color="red")
    plt.grid()
    plt.show()

'''
超参数设置:
'''
```

```python
showPlot = 0     #showPlot=1，显示最后的最佳路径图片
nc = 20          # 城市的个数
N = 100          # 染色体群体规模
MAXITER = 10**3 # 最大循环次数
SelfAdj = 0      #SelfAdj = 1时为自适应
randAccordDist = 0  #在mutate的时候根据位置值分配对应的概率，此概率决定哪个城市参与mutate

pc = 0.85        # 交叉概率
pw = 0.15        # 变异概率

npyfile='locations'+str(nc)+'.npy'
# 随机生成10个城市的坐标
# locations=np.asmatrix((np.random.rand(nc,2)*100).astype(np.compat.long))
# np.save(npyfile,locations)
locations=np.load(npyfile)

R = distCal(locations)       #计算出每个城市与其他各个城市之间的距离 R


# 步骤1，产生N个染色体的初始群体,保存在pop里面
pop = initPop(N,nc) #pop 遗传算法中的种群

iter=0
tstart=time()
while iter<MAXITER:
    iter = iter+1

    trajLength=calLen(pop,R)                #步骤2：计算每个染色体的路径长度

    fitness=calFitness(trajLength)          #       计算每个染色体的适应值
    b,f,ind=findBest(pop,fitness)        # 找到在当前种群中，适应度最高的个体

    # 步骤3：如果满足某些标准，算法停止
    if satified(fitness):
        break
    chooseNewP(pop,fitness)              #       否则，选取出一个新的群体
    crossPop(pop,pc,fitness,SelfAdj)
    #步骤4：交叉产生新染色体，得到新群体
    mutPop(pop,pw,fitness,SelfAdj)       #步骤5：基因变异
    pop[0,:] = b[0,:]                       #
    保留上一代中适应值最高的染色体

    if np.mod(iter,MAXITER/10)==1:
        titer=time()
        print("iter = %d after running %d seconds with route-length=%f" %(iter,
         int(titer-tstart),calLen4oneGene(b[0,:],R)))

#输出最优染色体/路径
print('nc=%d,N=%d, MAXITER=%d, pc=%f, pw=%f ' %(nc,N, MAXITER,pc ,pw))
print('最优的路径长度为：%f，适应值为：%f，对应的路径为：' %(calLen4oneGene(b[0
,:],R),f))
print(b[0,:])
```

参数设置

后面在计算过程中，只用到R

随机初始化

代的数

```python

#计算运行时间
tend=time()
print('The program need :  %f seconds.' %(tend-tstart))

#显示最优路径图像
if showPlot==1:
    drawTSP(locations, b[0,:])

```